

Chapter 2B: Solving Problems by Searching (Part II)

AMCS2104 Fundamentals of Artificial Intelligence

Content

- Search Algorithms' Performance Measurement
- Big-O Notation
- Exponential Complexity

Measuring Problem-Solving Performance

- An **algorithm's performance** can be **evaluated** and **compared** in 4 ways:
 - Completeness
 - Optimality
 - Time complexity
 - Space complexity

Measuring Problem-Solving Performance

□ Completeness

- **Question:** Is the algorithm guaranteed to find a solution when there is one and to correctly report failure when there is not?
- To be complete, a search algorithm must be **systematic** in the way it explores an **infinite state space**, making sure it can eventually reach any state that is connected to the initial state.

□ Optimality

- **Question:** Does it find a solution with the lowest path cost of all solutions?

Measuring Problem-Solving Performance

- **Time complexity and space complexity** are approximations of how much time and how much space an algorithm will take to process certain inputs respectively.
- **Time complexity**
 - **Question:** How long does it take to find a solution?
 - This can be measured in seconds, or more abstractly by the number of states and actions considered.
- **Space complexity**
 - **Question:** How much memory is needed to perform the search?

The Measurement of Complexity

- When a state space can be **represented explicitly**, the complexity is measured in terms of:
 - The number of vertices (state nodes) of the graph, and
 - The number of edges (distinct state/action pairs)
 - E.g., the map of Romania.

The Measurement of Complexity

- However, when a state space can only be **represented implicitly** by the initial state, actions, and transition model, the complexity is measured in terms of:
 - **b** , the branching factor or maximum number of successors of any node.
 - **d** , the depth of the shallowest goal node.
 - **m** , the maximum length (depth) of any path in the state space.

The Measurement of Complexity

Depth 0

Depth 1

Depth 2

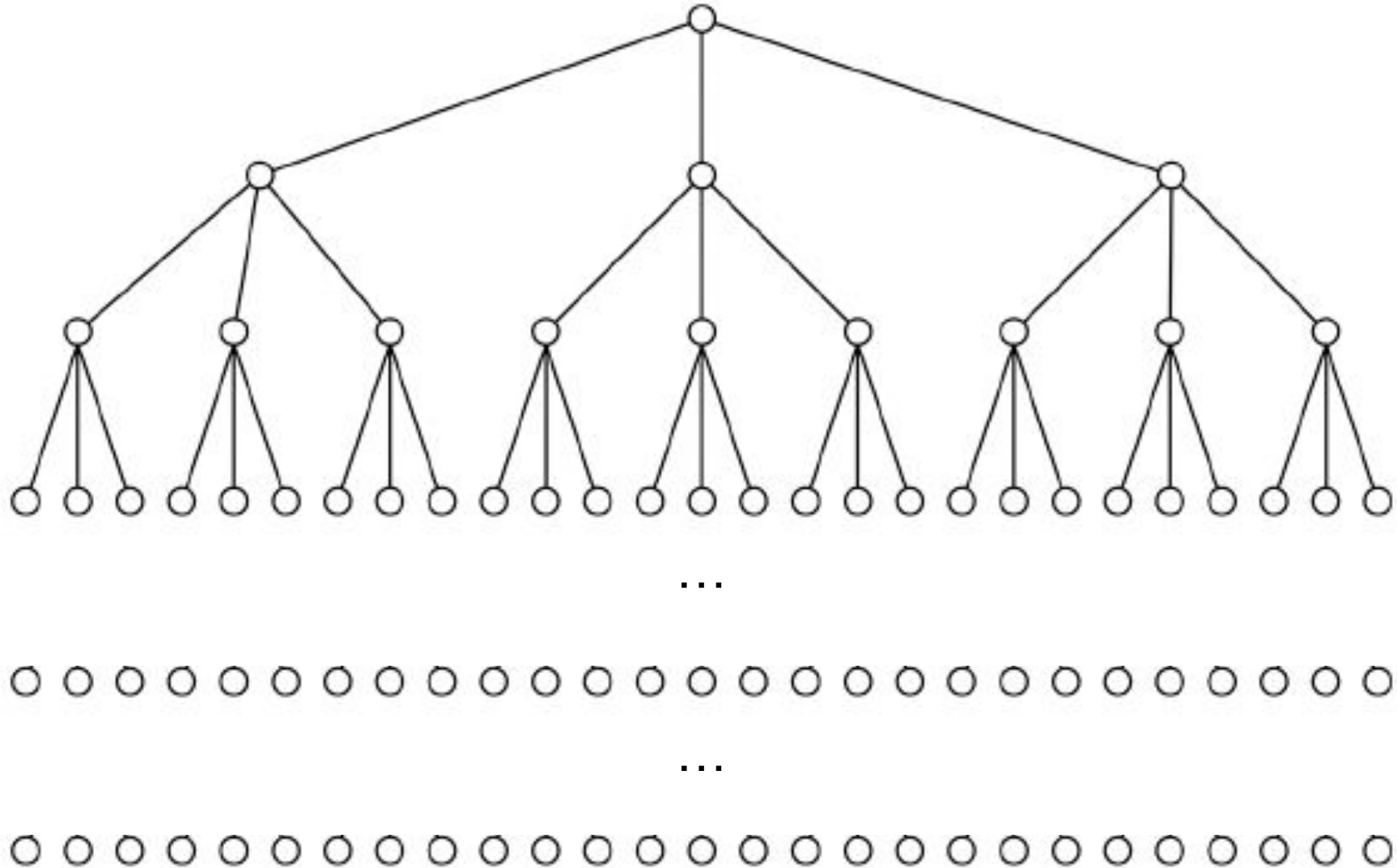
Depth 3

...

Depth d

...

Depth m



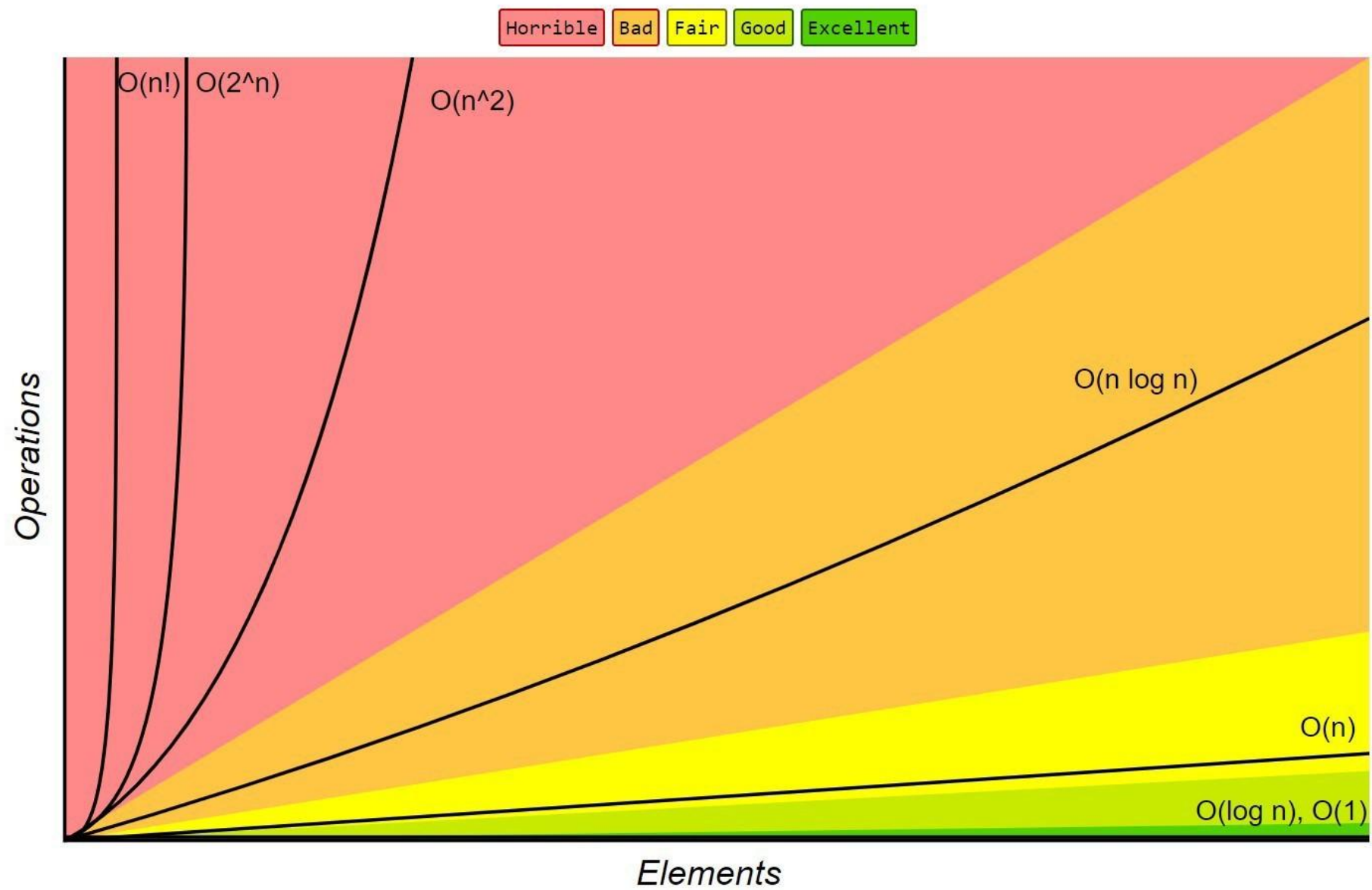
Big O-Notation

- **Big-O** (also known as **big-O notation**) is a metric for determining the **efficiency of an algorithm**.
- It doesn't tell how fast or how slow an algorithm will go but it measures how quickly the code grows relative to the input, as the input size increases.
- It is used to **expresses the complexity of an algorithm** by analyzing the **worst case scenario**.

Big O-Notation

- Constant time: $O(1)$
- Linear time: $O(n)$
- Logarithmic time: $O(n \log n)$
- Quadratic time: $O(n^2)$
- Exponential time: $O(2^n)$
- Factorial time: $O(n!)$

Big-O Complexity Chart



Exponential Complexity

□ **Question:** How scary an exponential complexity is?

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Time and memory requirements for breadth-first search.

The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

Chapter 3B: Uninformed Search (Part I)

AMCS2104 Fundamentals of Artificial Intelligence

Content

- Uninformed Search Strategies
- Breadth-First Search
- Uniform-Cost Search
- Depth-First Search

Uninformed Search Strategies

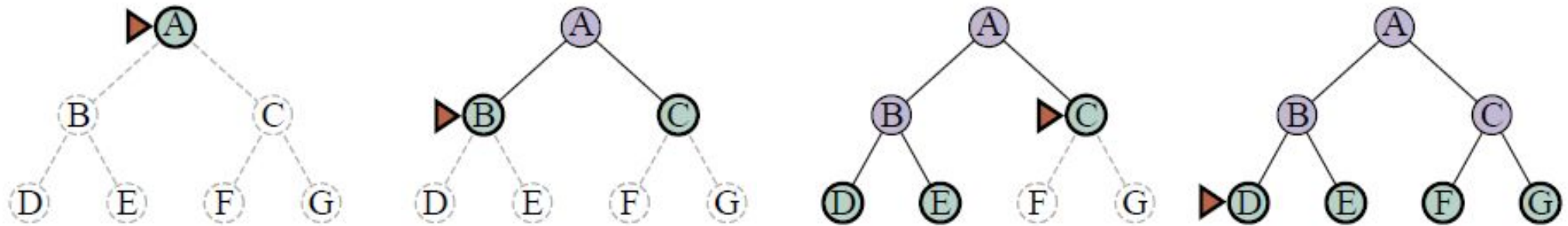
- Also called **blind search**.
- Has no additional information about states.
- Solely relies on the information provided in the problem definition.
- Only can differentiate between goal state and non-goal state.
- Cannot differentiate a non-goal state from another.

Breadth-First Search

- **Breadth-first search** is a simple strategy in which:
 - The **root node is expanded first**.
 - All the successors of the root node are expanded next.
 - Their successors are expanded next, and so on.
- All nodes at a given depth are expanded first before any nodes at the next level are expanded.

Breadth-First Search

- **Breadth-first search** is a simple strategy in which:
 - The **root node is expanded first**.
 - All the successors of the root node are expanded next.
 - Their successors are expanded next, and so on.



- All nodes at a given depth are expanded first before any nodes at the next level are expanded.

Breadth-First Search: Algorithm

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure  
  node  $\leftarrow$  NODE(problem.INITIAL)  
  if problem.IS-GOAL(node.STATE) then return node  
  frontier  $\leftarrow$  a FIFO queue, with node as an element  
  reached  $\leftarrow$  {problem.INITIAL}  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if problem.IS-GOAL(s) then return child  
      if s is not in reached then  
        add s to reached  
        add child to frontier  
  return failure
```

Breadth-First Search: Implementation

- Breadth-first search can be implemented easily using a **FIFO queue for the frontier**.
- The goal test applied in a breadth-first search is an **early goal test**.
- An **early goal test** checks whether a node is a solution as soon as it is generated.
- A **late goal test** only checks whether a node is a solution after it is popped off the queue.

Breadth-First Search: Performance Evaluation

□ Completeness

- Complete, provided **b** is finite, where **b** is the branching factor of the tree.

□ Optimality

- Cost-optimal, provided **all action costs are the same**.

□ Time complexity

- Total number of nodes generated: $1 + b^1 + b^2 + b^3 + \dots + b^d = O(b^d)$.
- Exponential complexity: $O(b^d)$.

□ Space complexity

- Exponential complexity: $O(b^d)$.

Time and Space Complexity

□ Analysis

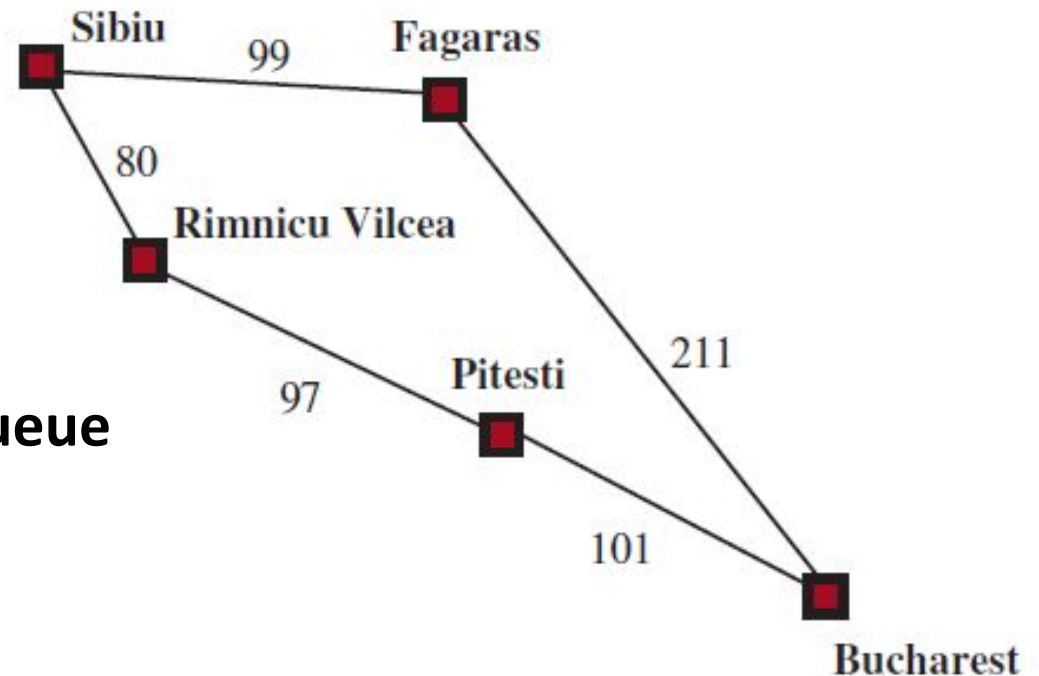
- Memory requirements are a bigger problem for breadth-first search than is the execution time.
- Yet, time is still a major factor.

□ Conclusion

- Exponential-complexity search problems cannot be solved by uninformed methods for any but the smallest instances.

Uniform-Cost Search

- The A.I. community named this algorithm as **uniform-cost search** but the **theoretical computer science community** refers to it as the **Dijkstra's algorithm**.
- Uniform-cost search expands a node n with **the lowest path cost, $g(n)$** , the cost of the path from the root to n .
- Frontier has to be stored as a **priority queue** ordered by function $g(n)$.



How Uniform-Cost Search Works

□ Step 1:

- The successors of Sibiu are Rimnicu Vilcea and Fagaras, with the path costs of **80** and **99**, respectively.

□ Step 2:

- The least-cost node, Rimnicu Vilcea, is expanded next, adding Pitesti with a path cost of **$80 + 97 = 177$** .

□ Step 3:

- The least-cost node is now Fagaras, so it is expanded, adding Bucharest with the path cost of **$99 + 211 = 310$** .

How Uniform-Cost Search Works

□ Step 4:

- A goal node has been generated, but the search keeps going.

□ Step 5:

- Pitesti is chosen for expansion and a second path to Bucharest with the path cost of **80 + 97 + 101 = 278** is added.

□ Step 6:

- The new path is better than the old one, the old one is replaced in **reached**.

□ Step 7:

- Bucharest, with the path cost of **278**, is selected for expansion and the solution is returned.

Uniform-Cost Search: Algorithm

function EXPAND(*problem, node*) **yields** nodes

$s \leftarrow node.STATE$

for each *action* **in** *problem.ACTIONS(s)* **do**

$s' \leftarrow problem.RESULT(s, action)$

$cost \leftarrow node.PATH-COST + problem.ACTION-COST(s, action, s')$

yield NODE(STATE= s' , PARENT=*node*, ACTION=*action*, PATH-COST= $cost$)

function UNIFORM-COST-SEARCH(*problem*) **returns** a solution node, or *failure*

return BEST-FIRST-SEARCH(*problem*, PATH-COST)

- The goal test applied in a uniform-cost search is a **late goal test**.

Uniform-Cost Search: Algorithm

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure  
  node  $\leftarrow$  NODE(STATE=problem.INITIAL)  
  frontier  $\leftarrow$  a priority queue ordered by f, with node as an element  
  reached  $\leftarrow$  a lookup table, with one entry with key problem.INITIAL and value node  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then  
        reached[s]  $\leftarrow$  child  
        add child to frontier  
  return failure
```

Uniform-Cost Search: Performance Evaluation

□ **Completeness**

- Complete if b is finite and all action costs exceed a small positive constant ϵ .

□ **Optimality**

- Cost-optimal.

□ **Time complexity**

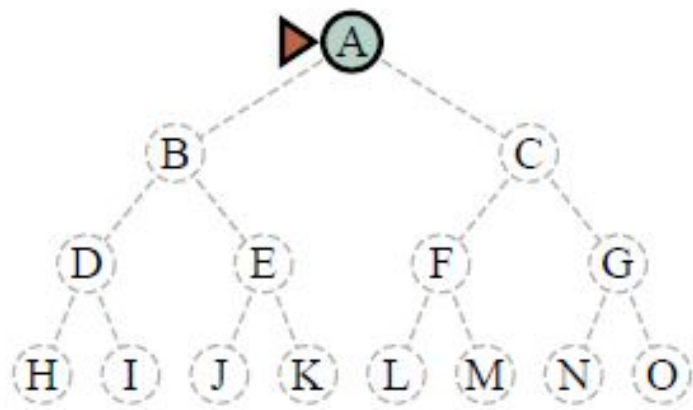
- Exponential complexity: $O(b^{1+(C^*/\epsilon)})$ where C^* is the cost of optimal solution.

□ **Space complexity**

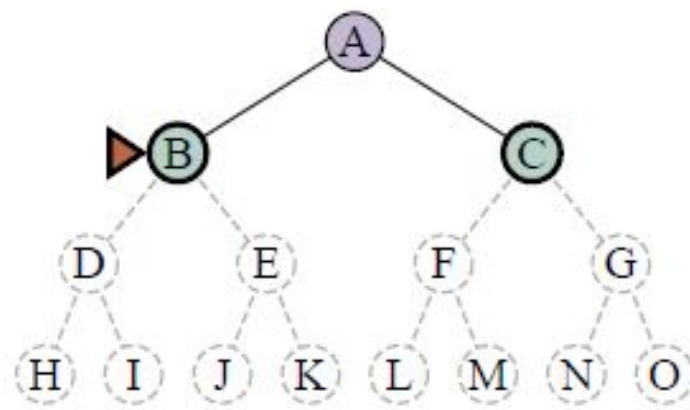
- Exponential complexity: $O(b^{1+(C^*/\epsilon)})$.

Depth-First Search

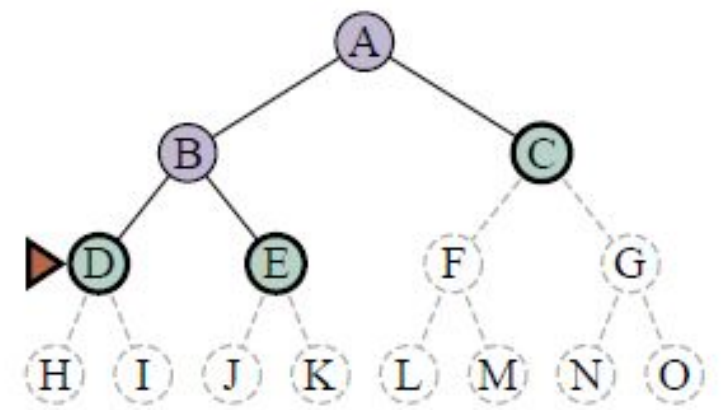
- **Depth-first search** always **expands the deepest node in the current frontier of the search tree**.
- The progress proceeds immediately to the deepest level of the search tree where the nodes have no successors (see Step 1 - 4).



Step 1



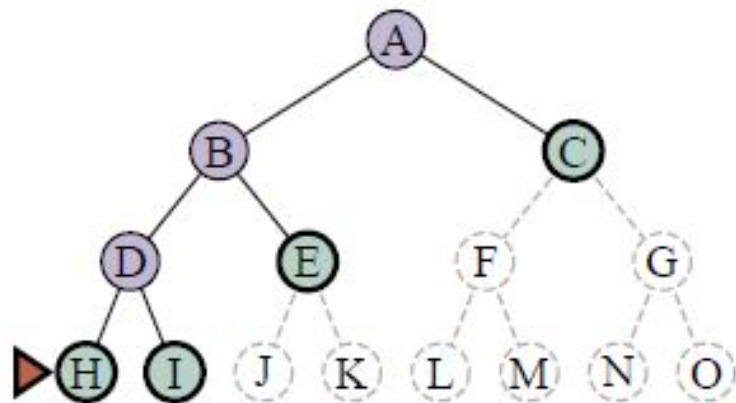
Step 2



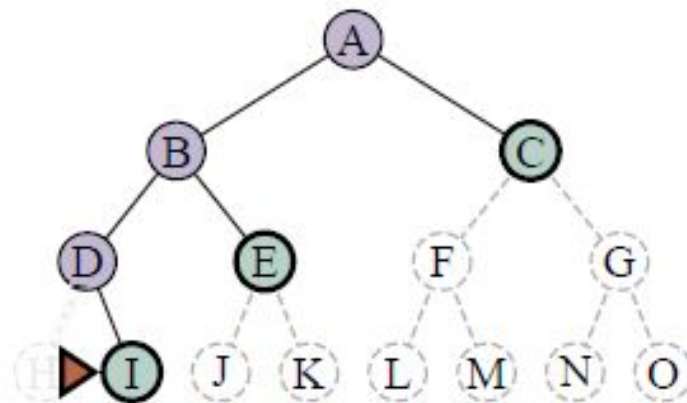
Step 3

Depth-First Search

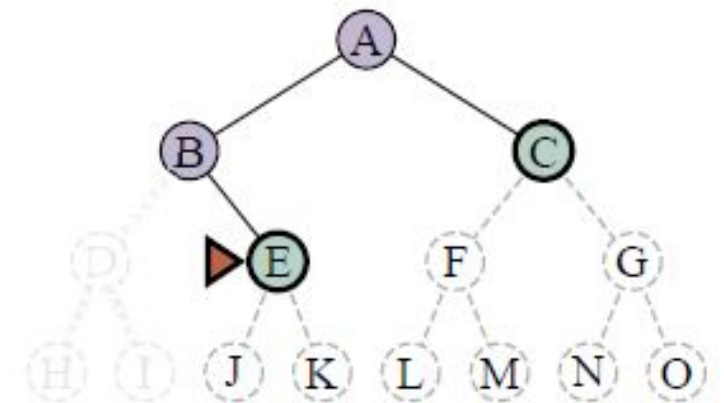
- It does not keep a table of **reached** states (see node **H** in Step 5).
- The search then **backs up** to the **next deepest node** that still has unexpanded successors (see node **E** in Step 6).
- Suppose node **M** is the goal node, the search works until Step 12.



Step 4

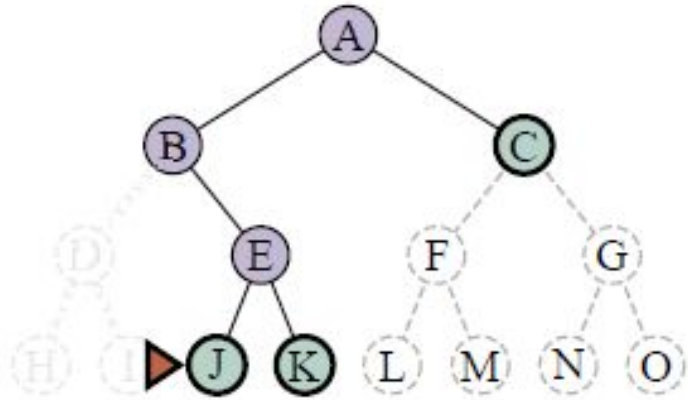


Step 5

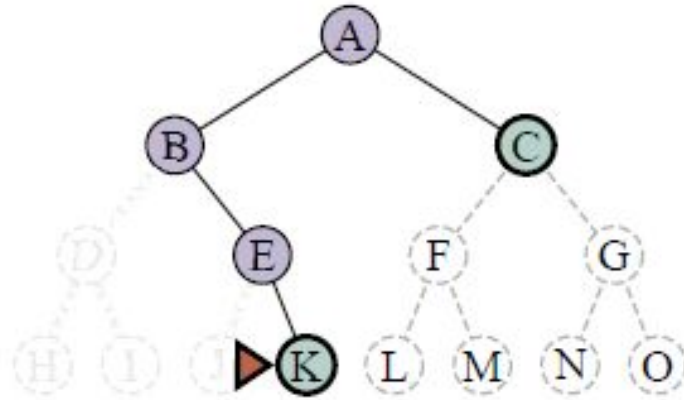


Step 6

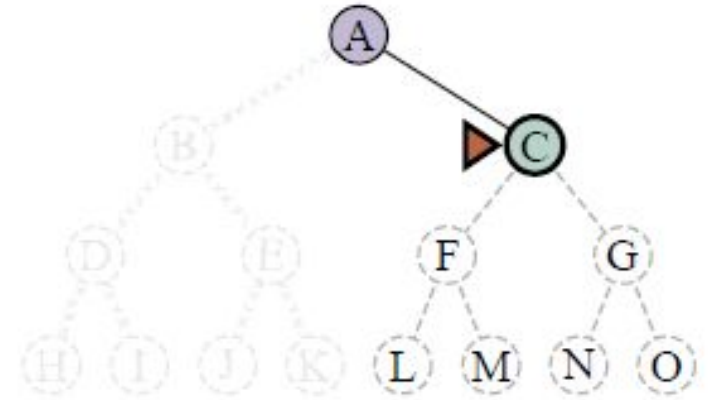
Depth-First Search



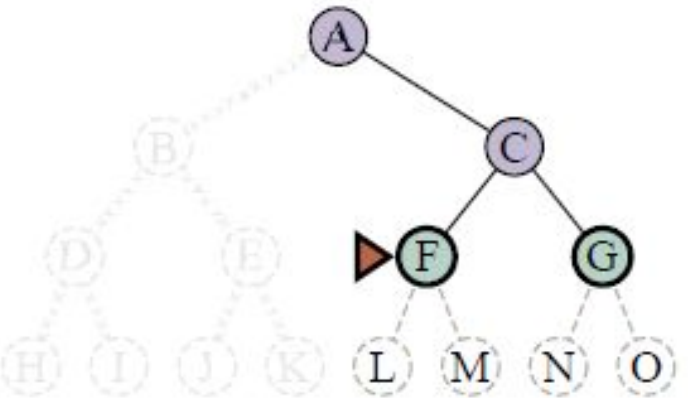
Step 7



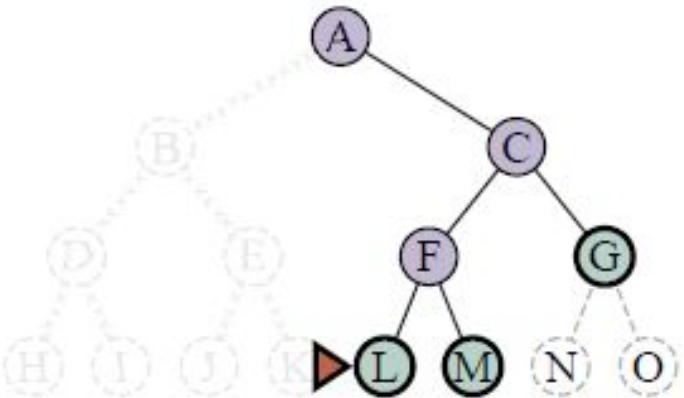
Step 8



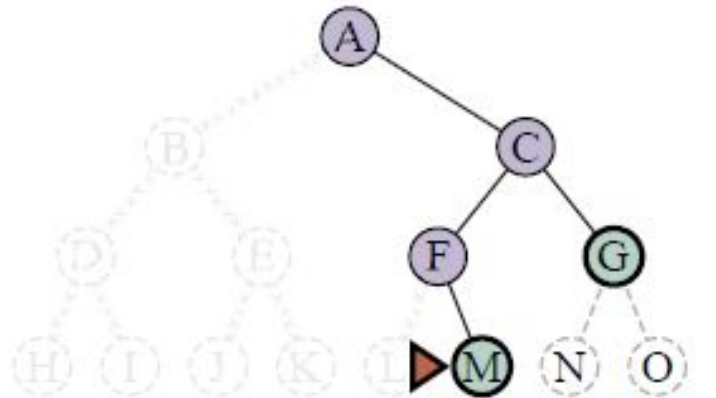
Step 9



Step 10



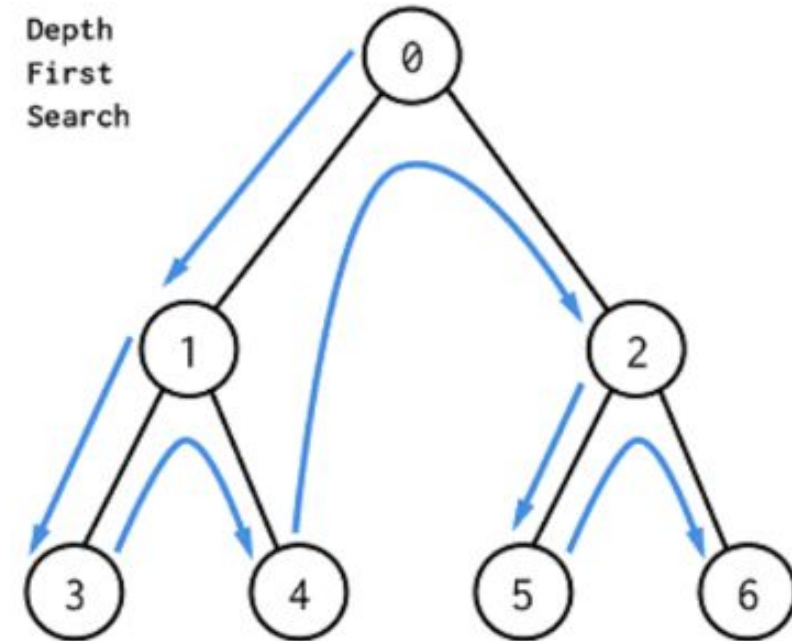
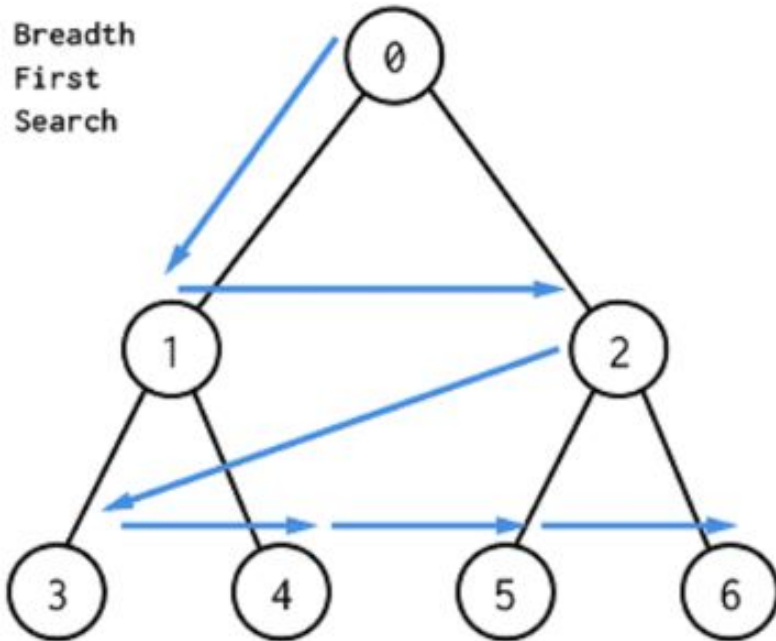
Step 11



Step 12

Depth-First Search: Implementation

- It is usually implemented as a **tree-like search**, but not graph search.
- Unlike breadth-first-search that uses a FIFO queue, it uses a **LIFO queue** in which the most recently generated node is chosen for expansion.



Depth-First Search: Performance Evaluation

□ **Completeness**

□ Complete

- Finite tree-shaped state spaces.
- Finite acyclic state spaces.
- Finite cyclic state spaces if it checks each new node for **cycles**.

□ Incomplete

- Infinite state spaces.

□ **Optimality**

- Not cost-optimal.

Depth-First Search: Performance Evaluation

□ Time complexity

- Exponential complexity: $O(b^m)$ where m is the maximum depth of the tree.

□ Space complexity

- Linear complexity: $O(bm)$.

Reading Homework

- **Artificial Intelligence: A Modern Approach**, 4th Edition, by Stuart Russell and Peter Norvig
 - Chapter 3: Solving Problems by Searching (page 93 to page 98)

The End